

Software Design Description 004 | LoRa Bench Test Protocol & Debug

Overview

This document describes the design of the LoRa bench test framework used to validate the RF link between the **ESP32-DevKitV1 (TX master)** and the **ESP32-S3 (echo slave)** prior to integrating the full coefficient packet pipeline.

It also documents four bugs responsible for a **100% packet loss regression**, their root causes, and the corrected design.

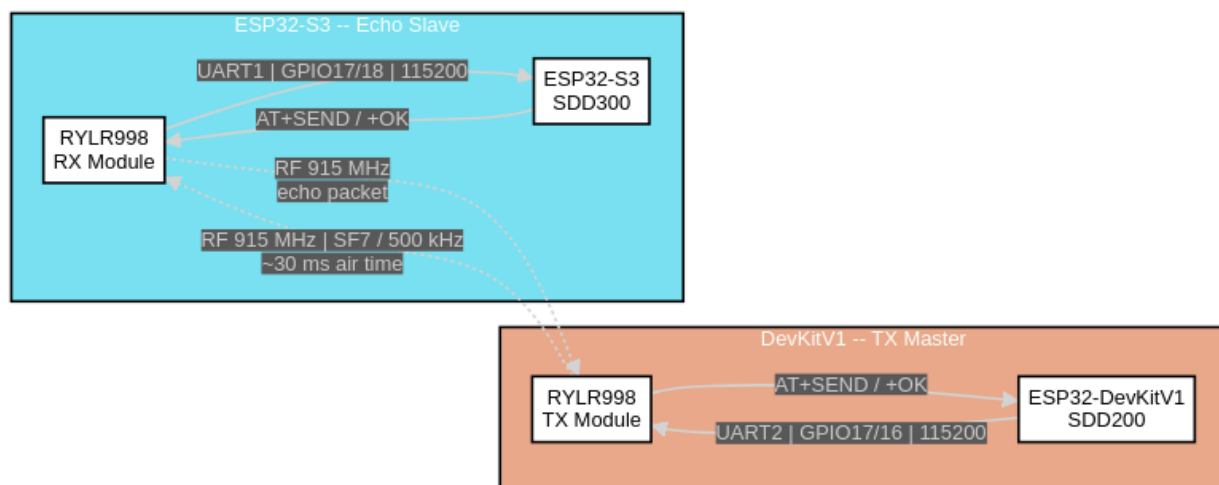
Hardware: REYAX RYLR998 x 2 | 915 MHz ISM band | AT-command over UART | Half-duplex LoRa

Table of Contents

- System Architecture
 - Hardware Setup
 - AT Command Protocol
 - Bench Test Design
 - State Machines
 - Root Cause Analysis
 - Corrected Data Flow
 - Configuration Reference
 - References
-

System Architecture

The bench test isolates the wireless link between the two ESP32 boards. All other subsystems (camera, FPGA, sensors, WiFi) are disabled by the `TEST_MODE_LORA_BENCH` build flag.



Hardware Setup

DevKitV1 UART → RYLR998 TX

Signal	ESP32 Pin	RYLR998 Pin	Direction
UART TX	GPIO17	RX	ESP32 → Module
UART RX	GPIO16	TX	Module → ESP32
UART Port	UART_NUM_2	—	—
Baud Rate	115200	—	8N1, no flow ctrl

ESP32-S3 UART → RYLR998 RX

Signal	ESP32 Pin	RYLR998 Pin	Direction
UART TX	GPIO17	RX	ESP32 → Module
UART RX	GPIO18	TX	Module → ESP32
UART Port	UART_NUM_1	—	—
Baud Rate	115200	—	8N1, no flow ctrl

Note: The S3 RX pin (GPIO18) differs from the DevKitV1 RX pin (GPIO16). Swapping TX/RX on the S3 silently kills all comms with no error output.

AT Command Protocol

The RYLR998 is not a transparent UART radio – it has its own AT command layer that must be understood to use the module correctly.

----- TRANSMIT SIDE -----

```
ESP32 --> AT+SEND=<dest>,<len>,<data>\r\n --> RYLR998
ESP32 <-- +OK\r\n <-- RYLR998
          (payload buffered; RF TX begins after +OK)
```

----- RECEIVE SIDE -----

```
RYLR998 --> +RCV=<src>,<len>,<data>,<rssi>,<snr>\r\n --> ESP32
```

Field definitions

Field	Description
<dest>	Destination module address (must match receiver's ADDRESS)
<src>	Source module address
<len>	Payload byte count (1–240)
<data>	Raw payload bytes
<rssi>	Received signal strength (dBm, negative)
<snr>	Signal-to-noise ratio (dB)

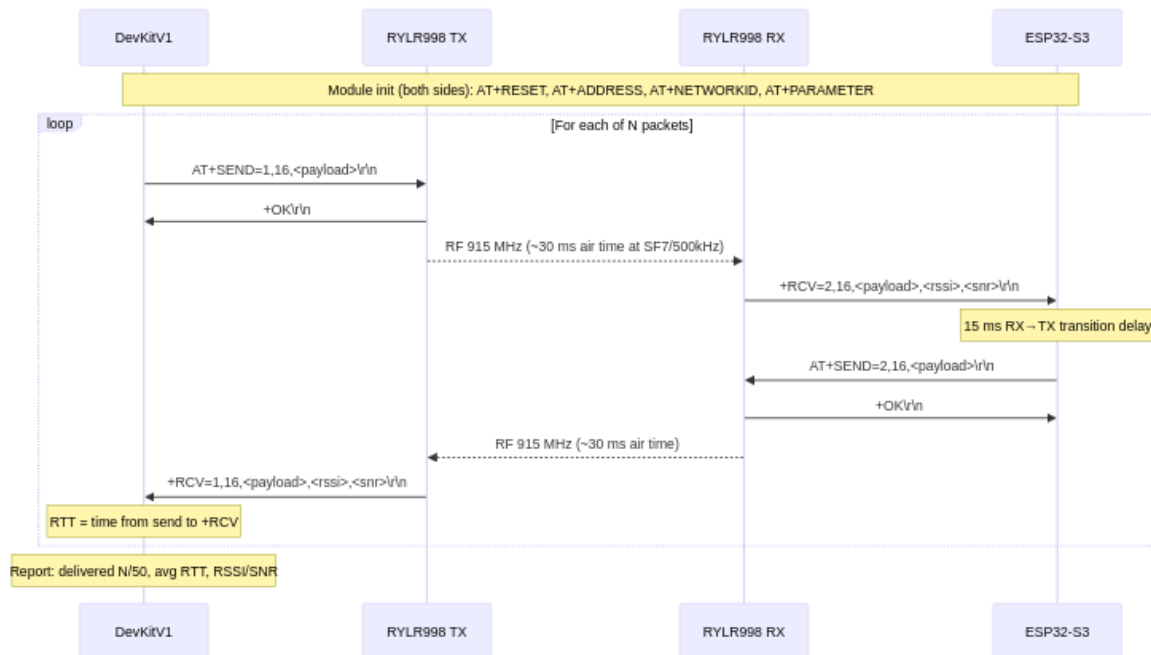
Initialization sequence (both modules)

```
AT+RESET      -> +RESET    (wait 1500 ms after reset)
AT+ADDRESS=<n> -> +OK
AT+NETWORKID=<n> -> +OK      <- both modules MUST share same ID
AT+PARAMETER=<SF>,<BW>,<CR>,<Preamble> -> +OK <- must match on both
AT+ADDRESS?    -> +ADDRESS=<n> (verify)
AT+NETWORKID?  -> +NETWORKID=<n> (verify)
AT+PARAMETER?  -> +PARAMETER=... (verify)
AT+BAND?       -> +BAND=... (verify)
```

Bench Test Design

The bench is a **request-response echo test**. The DevKitV1 is the master; the S3 is the slave. One packet is in-flight at a time.

Complete packet exchange sequence



Payload format

Each packet is BENCH_PKT_SIZE bytes (default 16):

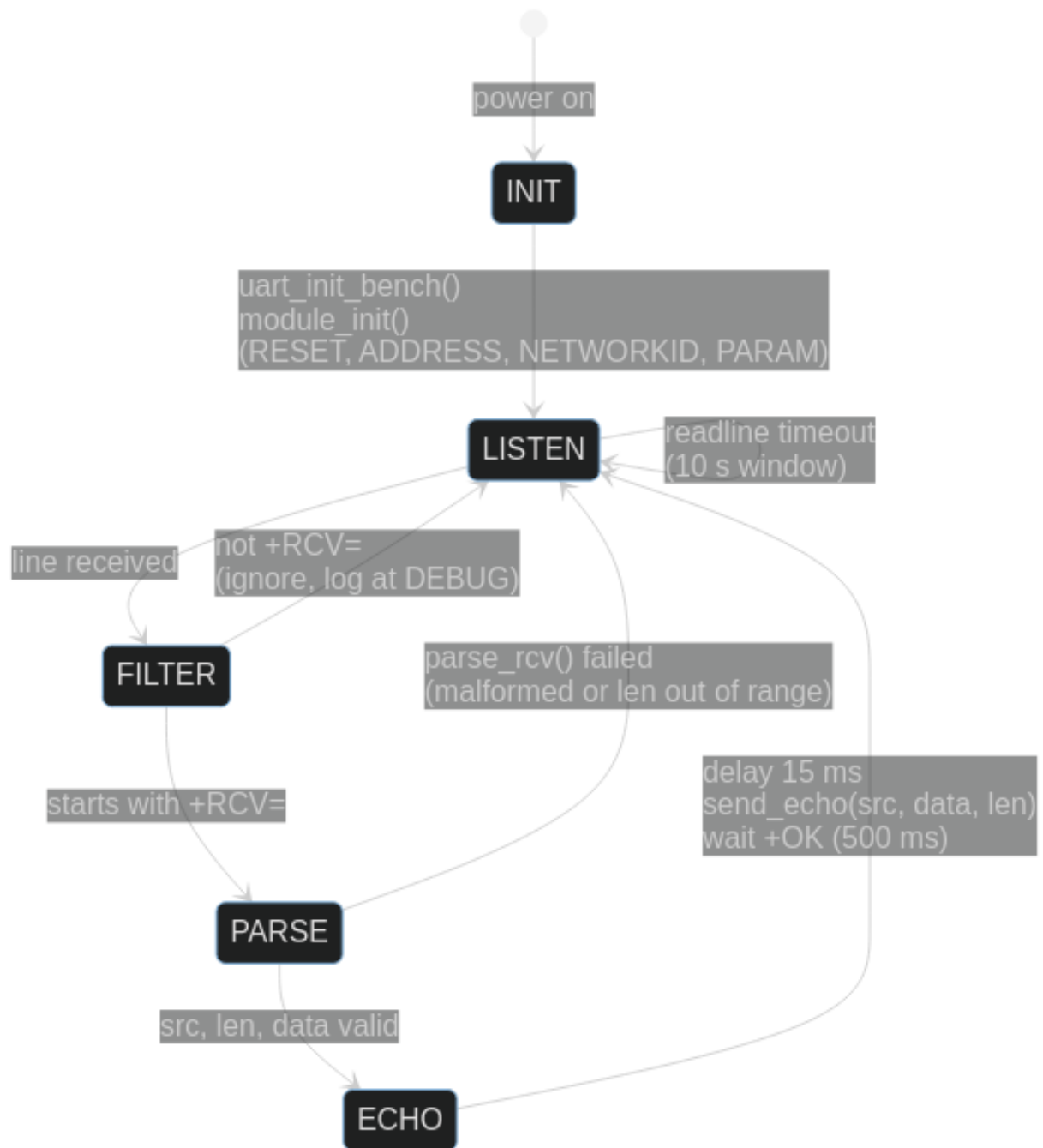
Bytes [0..7] : sequence number as 8-character lowercase hex ASCII e.g. "00000003"

Bytes [8..15] : padding byte 'A' (0x41) repeated

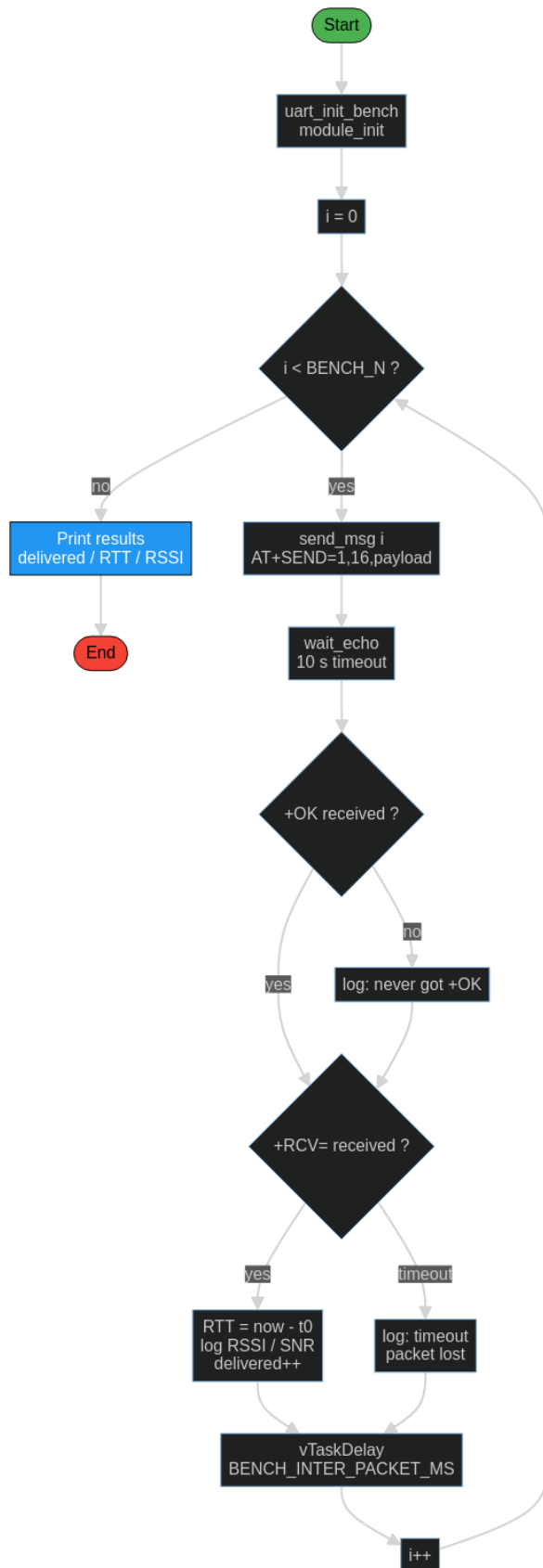
No null byte is placed in the wire payload. `snprintf` is used into a temporary buffer and then `memcpy`'d without the null terminator.

State Machines

S3 Echo Slave



DevKitV1 TX Master



Root Cause Analysis

Four bugs were found in the bench code. **Bugs 1 and 4 caused 100% packet loss.** Bugs 2 and 3 produced incorrect metrics.

Bug 1 – S3: UART driver delete/reinstall on every echo iteration (*root cause*)

File: esp/esp32s3_wroom/main/src/lora_bench.c lines 219–224

Broken code:

```
uart_driver_delete(LORA_UART_NUM);    // <- unregisters GPIO from UART peripheral
vTaskDelay(pdMS_TO_TICKS(10));
uart_init_bench();                    // <- reinstalls driver + 300 ms settle delay
vTaskDelay(pdMS_TO_TICKS(10));
send_echo(src, data, len);
```

Failure path:



Fix: Remove the reinit entirely. The UART driver should be installed once and left running. The RYLR998 only needs 15 ms to transition from receive to transmit mode.

// REMOVED: uart_driver_delete / uart_init_bench block

```
vTaskDelay(pdMS_TO_TICKS(15));    // RYLR998 RX->TX mode switch (per README)
send_echo(src, data, len);
```

Bug 2 – DevKitV1: Null byte embedded in wire payload

File: esp/esp32_devkitv1/main/src/interprocessor/lora_bench.c

```
// BROKEN -- snprintf writes '\0' at payload[8]:
memset(payload, 'A', size);
snprintf((char *)payload, 9, "%08lx", (unsigned long)seq);

// FIXED -- separate buffer, memcpy without null:
memset(payload, 'A', size);
char seq_str[9];
snprintf(seq_str, sizeof(seq_str), "%08lx", (unsigned long)seq);
memcpy(payload, seq_str, 8);
```

Some RYLR998 firmware versions treat the AT+SEND data field as a C string and truncate at the first \0. Sending a 16-byte payload with a null at byte 8 could cause the module to transmit only 8 bytes, causing the echo to arrive with len=8. The DevKitV1's RSSI parsing offset (hardcoded to skip BENCH_PKT_SIZE bytes) would then land in the wrong field.

Bug 3 – DevKitV1: RTT timestamps 2.4% high

File: esp/esp32_devkitv1/main/src/interprocessor/lora_bench.c

```
// BROKEN -- >> 10 divides by 1024, not 1000:
int64_t t0 = esp_timer_get_time() >> 10;
while ((esp_timer_get_time() >> 10) - t0 < timeout_ms)
int64_t rtt = (esp_timer_get_time() >> 10) - t0;
```

```
// FIXED -- true milliseconds:
int64_t t0 = esp_timer_get_time() / 1000;
while ((esp_timer_get_time() / 1000) - t0 < timeout_ms)
int64_t rtt = (esp_timer_get_time() / 1000) - t0;
```

`esp_timer_get_time()` returns microseconds. `>> 10` ($\div 1024$) gives units of 1024 μ s each. All RTT values and the timeout comparison used 1024 μ s as “1 ms,” producing results 2.4% higher than actual and an effective timeout of 10.24 s instead of 10.00 s.

Bug 4 – Startup race: DevKitV1 transmits before S3 is listening

Observed behavior:

DevKitV1 sends all 50 packets, receives +OK for each (local module accepted the AT command), but never receives a +RCV echo. 100% timeout. Meanwhile the S3 log shows it received only one packet — and that packet was **seq 2**, not seq 0. Packets 0 and 1 were transmitted over RF while the S3 was still in its post-init delay and not yet calling `readline`.

Why this happens:

The RYLR998 +OK response is **not a link-layer acknowledgment**. It means only that the local module accepted the AT+SEND command and will begin RF transmission. LoRa is a broadcast PHY — there is no pairing, no connection state, and no indication from the module that a remote receiver exists or is ready. The DevKitV1 has no way to know whether the S3 is listening.

Both boards boot independently. Their init sequences (`uart_init_bench` $\rightarrow$ 500 ms settle $\rightarrow$ `module_init` with AT+RESET $\rightarrow$ 1500 ms $\rightarrow$ AT config $\rightarrow$ 500 ms settle $\rightarrow$ `listening...`) take different wall-clock times depending on bootloader, flash size detection, and PSRAM init (S3 only). In the observed run:

```
DevKitV1 sends first packet: ~3184 ms after boot
S3 reaches listening state: ~3757 ms after boot
```

The S3 is ~573 ms late. At the bench test’s pace (~10.2 s per packet due to timeout + inter-packet delay), this only costs the first 1–2 packets. But those lost packets start the DevKitV1’s timeout cascade, and any secondary failure (USB disconnect, module lockup from an unread +RCV in the S3’s UART buffer) turns the rest of the trial into 100% loss.

Why a fixed delay is not the right fix:

Adding e.g. `vTaskDelay(pdMS_TO_TICKS(5000))` on the DevKitV1 before the send loop would paper over the race for bench-distance testing, but:

1. It’s fragile — the S3 boot time depends on flash size, PSRAM configuration, and any future init work. A change to either board’s startup could reintroduce the race.
2. It doesn’t scale — when this protocol moves off the bench and onto hardware with variable power-on sequencing (battery, solar, field deployment), there’s no guarantee which board powers up first.
3. It masks real failures — if the S3 fails to init its module entirely, the DevKitV1 will silently wait out the delay and then run a full 50-packet trial against a dead link.

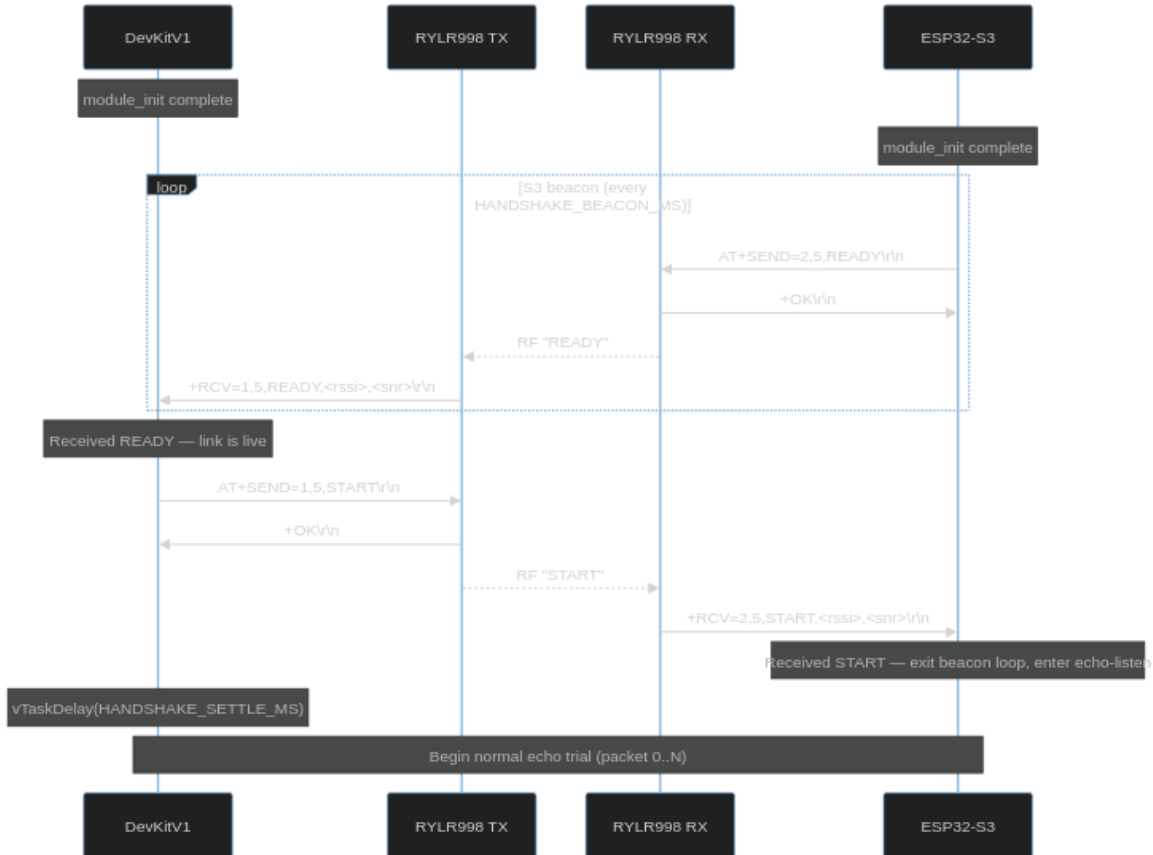
Fix: Application-level ready handshake

Since the RYLR998 provides no link-layer readiness signal, the application must implement one. The protocol is:

1. **S3 (echo slave)** completes `module_init`, then enters a beacon loop: it sends a short **READY** packet to the DevKitV1’s address every `HANDSHAKE_BEACON_MS` (e.g., 1000 ms) and listens for a **START** response between beacons.
2. **DevKitV1 (TX master)** completes `module_init`, then enters a handshake-wait loop: it calls `readline` with a short timeout, looking for a +RCV containing the **READY** payload. Upon receiving it, it sends a **START** packet back to the S3’s address.

3. **S3** receives the **START** packet (via **+RCV** during its listen window between beacons), exits the beacon loop, and enters the normal echo-listen state.
4. **DevKitV1** waits a short settle time after sending **START** (so the S3 has time to transition to listen mode), then begins the 50-packet trial.

If either board is not yet ready, the handshake simply hasn't completed — there is no timeout cascade, no wasted packets, and the trial only starts when both sides have confirmed link connectivity.



Handshake payload format

Packet	Sender	Payload	Length
READY	S3	READY	5
START	DevKitV1	START	5

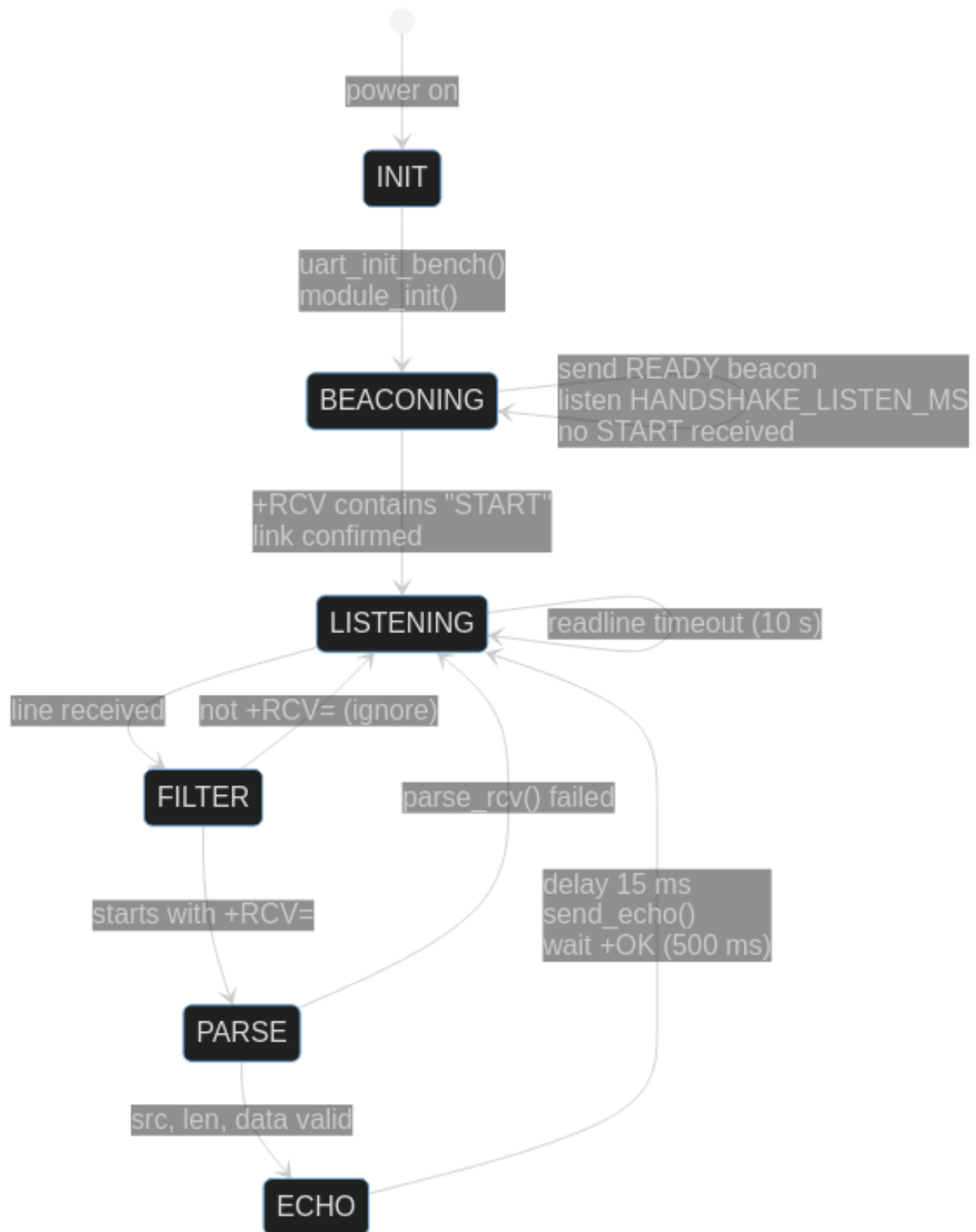
These are plain ASCII, no sequence number or framing needed. The `parse_rcv` function already extracts the data field — the handshake just checks `memcmp(data, "READY", 5)` or `memcmp(data, "START", 5)`.

Handshake configuration

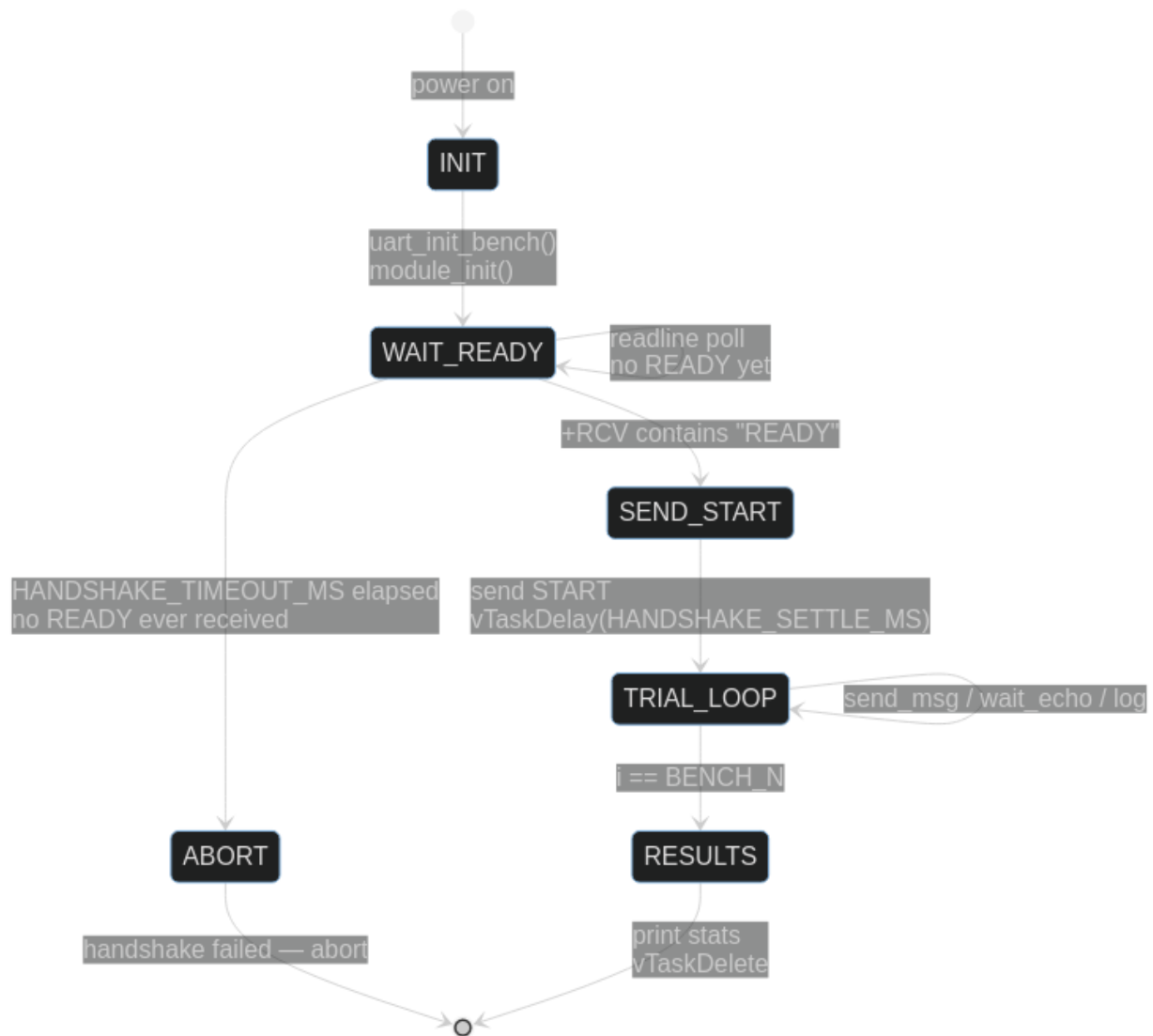
Macro	Value	Board	Notes
<code>HANDSHAKE_BEACON_MS</code>	1000	S3	Interval between READY beacons
<code>HANDSHAKE_LISTEN_MS</code>	500	S3	Listen window after each beacon for START
<code>HANDSHAKE_TIMEOUT_MS</code>	30000	DevKitV1	Max time to wait for READY before aborting

Macro	Value	Board	Notes
HANDSHAKE_SETTLE_MS	100	DevKitV1	Delay after sending START before first send

S3 handshake state machine

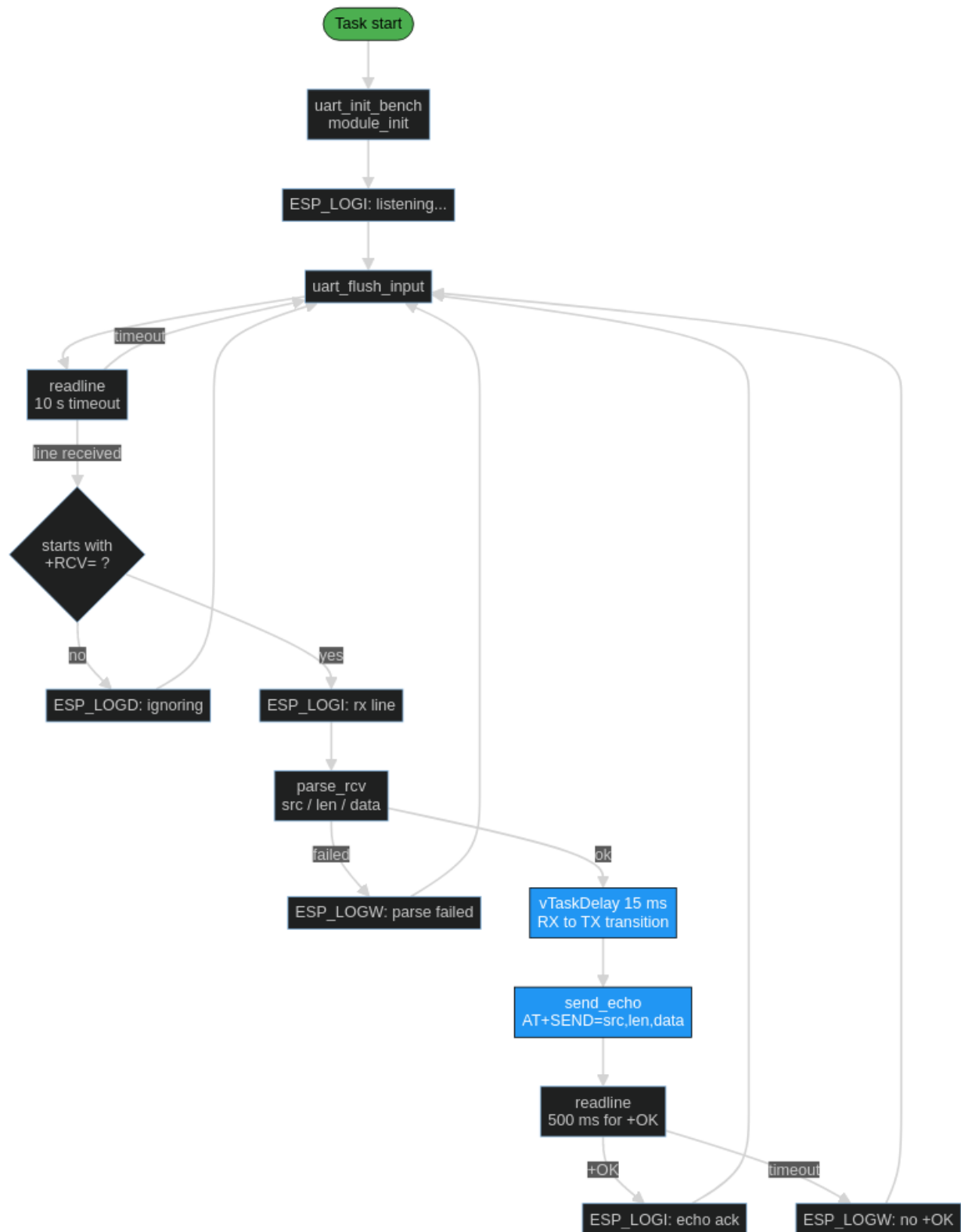


DevKitV1 handshake state machine



Corrected Data Flow

S3 echo slave – full corrected loop



Configuration Reference

Both `config.h` files must define matching RF parameters. Bench mode is gated by `#define TEST_MODE_LORA_BENCH` on both boards.

Macro	DevKitV1	(WROOM) S3	Notes
BENCH_SF	7	7	Spreading factor – must match
BENCH_BW	9	9	500 kHz – must match
BENCH_CR	1	1	Coding rate 4/5 – must match
BENCH_PREAMBLE	12	12	Preamble symbols – must match
LORA_NETWORK_ID	6	6	Network ID – must match
LORA_ADDRESS_SENDER	2	–	DevKitV1 module address
LORA_ADDRESS_RECEIVER	–	1	S3 module address
BENCH_DEST_ADDR	1	–	DevKitV1 sends to S3 addr 1
LORA_UART_NUM	UART_NUM_2	UART_NUM_1	Different peripheral per board
LORA_PIN_TX	GPIO17	GPIO17	ESP32 TX → Module RX
LORA_PIN_RX	GPIO16	GPIO18	Module TX → ESP32 RX
BENCH_PKT_SIZE	16	–	Bytes per packet
BENCH_N	50	–	Packets per trial
BENCH_TIMEOUT_MS	10000	–	Per-packet echo timeout (ms)
BENCH_INTER_PACKET_MS	10	–	Inter-packet gap (ms)
LORA_AT_TIMEOUT_MS	500	500	AT command response timeout (ms)

Expected bench results (bench distance)

Metric	Expected
Delivery	50 / 50 (100%)
Avg RTT	~160 ms
RSSI	~44 dBm
SNR	~13 dB

References

- SDD003 – LoRa Link Budget & Spreading Factor Calculations
- REYAX RYL998 AT Command Reference
- ESP-IDF UART Driver API

Source files modified by this SDD

File	Change
esp32s3_wroom/main/src/lora_bench.c	Remove UART reinit; add 15 ms delay; add READY b
esp32_devkitv1/main/src/interprocessor/lora_bench.c	Fix null byte; fix time units; simplify size clamp; add READ
esp32_devkitv1/main/inc/config.h	Add HANDSHAKE_TIMEOUT_MS, HAND
esp32s3_wroom/main/inc/config.h	Add HANDSHAKE_BEACON_MS, HAND